

Python para Economistas

Universidad Nacional de La Plata

Septiembre, 2022

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Funciones

Son objetos que sirven para modularizar el código, haciéndolo más reutilizable y fácil de leer, corregir y mantener.

Para crear una función comenzamos con 'def', luego un nombre para la función, luego entre () ponemos los argumentos y finalmente ':'

Funciones

Son objetos que sirven para modularizar el código, haciendolo más reutilizable y fácil de leer, corregir y mantener.

Para crear una función comenzamos con 'def', luego un nombre para la función, luego entre () ponemos los argumentos y finalmente ':'

```
1 >>> def gdp_per_capita(gdp, poblacion):  
2 >>>     gdp_pc = round(gdp/poblacion*1000)  
3 >>>     return gdp_pc
```

Funciones

Son objetos que sirven para modularizar el código, haciéndolo más reutilizable y fácil de leer, corregir y mantener.

Para crear una función comenzamos con 'def', luego un nombre para la función, luego entre () ponemos los argumentos y finalmente ':'

```
1 >>> def gdp_per_capita(gdp, poblacion):  
2 >>>     gdp_pc = round(gdp/poblacion*1000)  
3 >>>     return gdp_pc
```

Para usar una función escribimos el nombre y entre () ponemos los argumentos

```
1 >>> gdp_per_capita(77732, 3462)  
2 22453  
3 >>> gdp_per_capita(ury_carac['gdp'], ury_carac['pob'])  
4 22453
```

Los atributos de las funciones pueden tener un valor predefinido o no.

Los atributos de las funciones pueden tener un valor predefinido o no.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False):
2 >>>     if gdp_billion: #equivale a if gdp_billion==True:
3 >>>         gdp_pc = round(gdp*1000000/pob)
4 >>>     else:
5 >>>         gdp_pc = round(gdp/pob)
6 >>>     return gdp_pc
```


Los atributos de las funciones pueden tener un valor predefinido o no.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False):
2 >>>     if gdp_billion: #equivale a if gdp_billion==True:
3 >>>         gdp_pc = round(gdp*1000000/pob)
4 >>>     else:
5 >>>         gdp_pc = round(gdp/pob)
6 >>>     return gdp_pc
```

Para usar esta función escribimos el nombre y entre () ponemos los argumentos

Los atributos de las funciones pueden tener un valor predefinido o no.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False):
2 >>>     if gdp_billion: #equivale a if gdp_billion==True:
3 >>>         gdp_pc = round(gdp*1000000/pob)
4 >>>     else:
5 >>>         gdp_pc = round(gdp/pob)
6 >>>     return gdp_pc
```

Para usar esta función escribimos el nombre y entre () ponemos los argumentos

```
1 >>> print(gdp_per_capita(445445000000, 44938712))
2 9912.27
3 >>> print(gdp_per_capita(445445, 44938712, True))
4 9912.27
```

Los valores predefinidos generan que a veces no sea suficiente con especificar los valores de los atributos solo por la posición.

Funciones

Los valores predefinidos generan que a veces no sea suficiente con especificar los valores de los atributos solo por la posición.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False,
2                          gdp_millon=False):
3 >>>     if gdp_billion: #equivale a if gdp_billion==True:
4 >>>         gdp_pc = round(gdp*1000000/pob)
5 >>>     elif gdp_millon:
6 >>>         gdp_pc = round(gdp*1000/pob)
7 >>>     else:
8 >>>         gdp_pc = round(gdp/pob)
9 >>>     return gdp_pc
```

Funciones

Los valores predefinidos generan que a veces no sea suficiente con especificar los valores de los atributos solo por la posición.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False,
2                          gdp_millon=False):
3 >>>     if gdp_billion: #equivale a if gdp_billion==True:
4 >>>         gdp_pc = round(gdp*1000000/pob)
5 >>>     elif gdp_millon:
6 >>>         gdp_pc = round(gdp*1000/pob)
7 >>>     else:
8 >>>         gdp_pc = round(gdp/pob)
9 >>>     return gdp_pc

1 >>> print(gdp_per_capita(445445000000, 44938712))
2 9912.27
3 >>> print(gdp_per_capita(445445, 44938712, True, False))
4 9912.27
5 >>> print(gdp_per_capita(445445, 44938712, gdp_millon=True))
6 9912.27
```

Funciones: DocString

Un `DocString` es una de cadena de caracteres (`strings`) o varias líneas, delimitado por comillas al comienzo de un módulo, clase, método o función que describe lo que hace la función.

Funciones: DocString

Un DocString es una de cadena de caracteres (**strings**) o varias líneas, delimitado por comillas al comienzo de un módulo, clase, método o función que describe lo que hace la función.

```
1 >>> def gdp_per_capita(gdp, poblacion):
2 >>>     """
3 >>>     Esta funcion calcula el GDP per Capita
4 >>>     Input:
5 >>>         gdp(int): el GDP de un pais x
6 >>>         poblacion(int): la poblacion de un pais x
7 >>>     Output:
8 >>>         gdp_pc(int): el gdp per capita del pais x
9 >>>     """
10 >>>     gdp_pc = round(gdp/poblacion*1000)
11 >>>     return gdp_pc

1 >>> help(gdp_per_capita)
```

Funciones: Pregunta

Definamos un diccionario con la información de algunos países:

```
1 >>> {'argentina': {'cod': 'ARG', 'pob': 44939, 'gdp':  
1031215}, 'bolivia': {'cod': 'BOL', 'pob': 11513, 'gdp':  
104609}, 'brasil': {'cod': 'BRA', 'pob': 211050, 'gdp':  
3220373}, 'chile': {'cod': 'CHL', 'pob': 18952, 'gdp':  
476738}, 'paraguay': {'cod': 'PRY', 'pob': 7045, 'gdp':  
93062}, 'uruguay': {'cod': 'URY', 'pob': 3462, 'gdp':  
77732}}
```


Funciones: Pregunta

Definamos un diccionario con la información de algunos países:

```
1 >>> {'argentina': {'cod': 'ARG', 'pob': 44939, 'gdp':  
1031215}, 'bolivia': {'cod': 'BOL', 'pob': 11513, 'gdp':  
104609}, 'brasil': {'cod': 'BRA', 'pob': 211050, 'gdp':  
3220373}, 'chile': {'cod': 'CHL', 'pob': 18952, 'gdp':  
476738}, 'paraguay': {'cod': 'PRY', 'pob': 7045, 'gdp':  
93062}, 'uruguay': {'cod': 'URY', 'pob': 3462, 'gdp':  
77732}}
```

1) ¿Cómo usarían la función `gdp_per_capita` para agregar el gdp per capita a cada país del diccionario `'países_info'`?

Funciones: Pregunta

Algo muy util es pensar en pseudo código y despues llenar las lineas de codigo (googleando):

Funciones: Pregunta

Algo muy útil es pensar en pseudo código y después llenar las líneas de código (googleando):

```
1 >>> # Hacer un loop para iterar por los países del
    diccionario maestro. Que el loop me de la llave (el país)
    y el valor (el dic interno)
2 >>> #Al diccionario interno crearle un nuevo par de llave y
    valor. Que la llave sea 'gdp_pc' y que el valor sea el
    resultado de la función gdp_per_capita. A la función le
    doy los valores que el diccionario interno tienen
    guardados con las llaves 'gdp' y 'pob'
3
4 >>> paises_info
5 {'argentina': {'cod': 'ARG', 'pob': 44939, 'gdp': 1031215, '
    gdp_pc': 22947}, 'bolivia': {'cod': 'BOL', 'pob': 11513,
    'gdp': 104609, 'gdp_pc': 9086}, 'brasil': {'cod': 'BRA'
    , 'pob': 211050, 'gdp': 3220373, 'gdp_pc': 15259}, '
    chile': {'cod': 'CHL', 'pob': 18952, 'gdp': 476738, '
    gdp_pc': 25155}, 'paraguay': {'cod': 'PRY', 'pob': 7045,
    'gdp': 93062, 'gdp_pc': 13210}, 'uruguay': {'cod': 'URY'
    , 'pob': 3462, 'gdp': 77732, 'gdp_pc': 22453}}
```

Funciones: Ámbito interno vs. ámbito global

Ámbito interno: contiene las variables locales (por ejemplo, de una función)

Ámbito global: contiene las variables globales del módulo actual.

```
1 >>> a = 3 #a definida en ambito global
2 >>>
3 >>> def suma_uno(numero):
4 ...     a = numero + 1 #a definida en ambito interno
5 ...     print(a)
6 >>>
```

Funciones: Ámbito interno vs. ámbito global

Ámbito interno: contiene las variables locales (por ejemplo, de una función)

Ámbito global: contiene las variables globales del módulo actual.

```
1 >>> a = 3 #a definida en ambito global
2 >>>
3 >>> def suma_uno(numero):
4 ...     a = numero + 1 #a definida en ambito interno
5 ...     print(a)
6 >>>
```

```
1 >>> suma_uno(10)
2 11
```

Funciones: Ámbito interno vs. ámbito global

Ámbito interno: contiene las variables locales (por ejemplo, de una función)

Ámbito global: contiene las variables globales del módulo actual.

```
1 >>> a = 3 #a definida en ambito global
2 >>>
3 >>> def suma_uno(numero):
4 ...     a = numero + 1 #a definida en ambito interno
5 ...     print(a)
6 >>>
```

```
1 >>> suma_uno(10)
2 11
```

```
1 >>> print(a)
2 3
```

Es decir, el ámbito interno de la función 'tapa' al ámbito global.

Ámbito: Pregunta

2) Si corremos el siguiente bloque de código ¿Cuántos print() se ejecutarán? ¿Qué valores devolverá cada uno de esos print()? (escribalos en el orden que saldrían)

```
1 >>> gdp_arg = 1031215
2 >>> gdp_u ru = 77732
3 >>> gdp_regional = gdp_arg + gdp_u ru
4
5 >>> def suma_gdp(gdp_regional, gdp_pais):
6 ...     gdp_regional += gdp_pais
7 ...     print(gdp_regional)
8 >>>
9 >>> print(gdp_regional)
10 >>> gdp_pry = 93062
11 >>>
12 >>> suma_gdp(gdp_regional, gdp_pry)
13 >>> print(gdp_regional)
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Manejo de Errores

Al momento de aprender es bueno cometer errores e ir leyendo los motivos de estos errores.

Manejo de Errores

Al momento de aprender es bueno cometer errores e ir leyendo los motivos de estos errores.

```
1 >>> def suma_uno(num):
2     ...     '''
3     ...     Recibe un numero y devuelve ese numero m s uno
4     ...     '''
5     ...     return num + 1
6 >>>
```

```
1 >>> suma_uno(1)
2 2
```

```
1 >>> mi_lista = [1, 20, '30']
2 >>>
3 >>> for elemento in mi_lista:
4     ...     suma_uno(elemento)
5 ?
```

Manejo de Errores

```
1 >>> mi_lista = [1, 20, '30']
2 >>>
3 >>> for elemento in mi_lista:
4 ...     suma_uno(elemento)
5 -----
6 TypeError                                Traceback (most recent call last)
7 <ipython-input-40-aa79ec4ee6f3> in <module>
8     1 for elemento in mi_lista:
9 ----> 2     suma_uno(elemento)
10 <ipython-input-39-adb5a9d88b44> in suma_uno(num)
11     3 def suma_uno(num):
12 ----> 4     return num + 1
13 TypeError: can only concatenate str (not "int") to str
```

Manejo de Errores

Analizar el código por partes, comentar (con #) las líneas con conflicto y usar print() son grandes estrategias en esta etapa.

```
1 >>> for elemento in mi_lista:
2 ...     #suma_uno(elemento)
3 ...     print("item:", elemento)
4 ...     print("tipo:", type(elemento))
```

```
1 item: 1
2 tipo: <class 'int'>
3 item: 2
4 tipo: <class 'int'>
5 item: 30
6 tipo: <class 'str'>
```

Manejo de Errores

Además, en python existen mecanismos para actuar en caso de comportamiento erróneo o inesperado. Para esto, se utiliza la declaración `try/except`:

```
1 >>> for elem in lista:
2 ...     try:
3 ...         suma_uno(elem)
4 ...     except:
5 ...         print("No se pudo sumar", elem, ". Es", type(elem))
```

```
1 >>> suma_uno(mi_lista)
2 2
3 3
4 No se pudo sumar 30. Es <class 'str'>.
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos**
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Módulos

Un módulo es un archivo de Python (con extensión `.py`) que puede contener definiciones de funciones o sentencias ejecutables, y puede ser *importado* desde otros módulos o desde una sesión interactiva (y en particular, desde una sesión de Jupyter).

Un módulo puede ser escrito por el usuario, y también pueden utilizarse módulos escritos por otras personas (previa instalación de los mismos).

Módulos

Un módulo es un archivo de Python (con extensión `.py`) que puede contener definiciones de funciones o sentencias ejecutables, y puede ser *importado* desde otros módulos o desde una sesión interactiva (y en particular, desde una sesión de Jupyter).

Un módulo puede ser escrito por el usuario, y también pueden utilizarse módulos escritos por otras personas (previa instalación de los mismos).

Permiten ampliar notablemente las funcionalidades nativas de Python, y también almacenar código que uno puede querer reutilizar en el futuro (un Jupyter notebook también permite esto último, pero los notebooks no son importables de manera directa).

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- `math`: para funciones matemáticas.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- `math`: para funciones matemáticas.
- `os`: para acceder a funcionalidades del sistema operativo.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- **math**: para funciones matemáticas.
- **os**: para acceder a funcionalidades del sistema operativo.
- **re**: para manejo de expresiones regulares.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- **math**: para funciones matemáticas.
- **os**: para acceder a funcionalidades del sistema operativo.
- **re**: para manejo de expresiones regulares.
- **csv**: para manejo de archivos tabulares separados por comas.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- `math`: para funciones matemáticas.
- `os`: para acceder a funcionalidades del sistema operativo.
- `re`: para manejo de expresiones regulares.
- `csv`: para manejo de archivos tabulares separados por comas.
- `pickle`: para almacenar objetos de Python en el disco, y leerlos.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- **math**: para funciones matemáticas.
- **os**: para acceder a funcionalidades del sistema operativo.
- **re**: para manejo de expresiones regulares.
- **csv**: para manejo de archivos tabulares separados por comas.
- **pickle**: para almacenar objetos de Python en el disco, y leerlos.

La lista completa se puede consultar [acá](#).

¿Cómo importamos un módulo?

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹.

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

```
1 >>> archivo = csv.reader("archivo.csv")
```

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

```
1 >>> archivo = csv.reader("archivo.csv")
```

Se le puede asignar un alias al módulo al momento de importarlo:

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

```
1 >>> archivo = csv.reader("archivo.csv")
```

Se le puede asignar un alias al módulo al momento de importarlo:

```
1 >>> import csv as comma_separated_values
2 >>> archivo = comma_separated_values.reader("archivo.csv")
```

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

Es decir, de esta manera *no necesitamos* usar el nombre del módulo (`csv`) para acceder a su atributo (`reader`).

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

Es decir, de esta manera *no necesitamos* usar el nombre del módulo (`csv`) para acceder a su atributo (`reader`).

También se puede usar un alias para el atributo:

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

Es decir, de esta manera *no necesitamos* usar el nombre del módulo (`csv`) para acceder a su atributo (`reader`).

También se puede usar un alias para el atributo:

```
1 from csv import reader as lector
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos**
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Clases y Objetos

Las *clases* son tipos de datos que permiten representar una entidad encapsulando los datos y funcionalidad asociados a ella. Los *objetos* son instancias de estas clases.

Clases y Objetos

Las *clases* son tipos de datos que permiten representar una entidad encapsulando los datos y funcionalidad asociados a ella. Los *objetos* son instancias de estas clases.

Las clases pueden ser definidas por el usuario, y es la forma de crear nuevos tipos de datos más complejos en base a los tipos nativos de Python.

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

```
1 class Persona():  
2     dni = 57000000  
3     pasaporte = "ABN" + str(dni)
```

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

```
1 class Persona():  
2     dni = 57000000  
3     pasaporte = "ABN" + str(dni)
```

Luego de definir la clase, podemos acceder a sus atributos con un `."`:

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
```

Luego de definir la clase, podemos acceder a sus atributos con un `."`:

```
1 >>> Persona.dni
2 57000000
3 >>> Persona.pasaporte
4 ABN57000000
```

Clases como "fábricas" de objetos

Las clases permiten definir una función especial llamada *constructor*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def __init__(self, nombre, apellido, edad, sexo):
6         self.nombre = nombre
7         self.apellido = apellido
8         self.edad = edad
9         self.sexo = sexo
```

Clases como "fábricas" de objetos

Las clases permiten definir una función especial llamada *constructor*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def __init__(self, nombre, apellido, edad, sexo):
6         self.nombre = nombre
7         self.apellido = apellido
8         self.edad = edad
9         self.sexo = sexo
```

Una vez definida la clase, el constructor se invoca usando el nombre de la clase y produce un *objeto* de dicha clase:

Clases como "fábricas" de objetos

Las clases permiten definir una función especial llamada *constructor*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def __init__(self, nombre, apellido, edad, sexo):
6         self.nombre = nombre
7         self.apellido = apellido
8         self.edad = edad
9         self.sexo = sexo
```

Una vez definida la clase, el constructor se invoca usando el nombre de la clase y produce un *objeto* de dicha clase:

```
1 >>> juan = Persona("Juan", "Tarufetti", 76, "M")
2 >>> lucia = Persona("Lucia", "Varela", 45, "F")
3
4 >>> lucia.apellido
5 "Varela"
```

Métodos de clases

Las clases permiten no sólo encapsular los datos relativos a un objeto, sino también su comportamiento. Esto se hace a través de funciones, sólo que dentro de una clase se llaman *métodos*:

Métodos de clases

Las clases permiten no sólo encapsular los datos relativos a un objeto, sino también su comportamiento. Esto se hace a través de funciones, sólo que dentro de una clase se llaman *métodos*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def cambiosexo(self, nuevosexo):
6         self.sexo = nuevosexo
7
8     def __init__(self, nombre, apellido, edad, sexo):
9         self.nombre = nombre
10        self.apellido = apellido
11        self.edad = edad
12        self.sexo = sexo
```

Métodos de clases

Las clases permiten no sólo encapsular los datos relativos a un objeto, sino también su comportamiento. Esto se hace a través de funciones, sólo que dentro de una clase se llaman *métodos*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def cambiosexo(self, nuevosexo):
6         self.sexo = nuevosexo
7
8     def __init__(self, nombre, apellido, edad, sexo):
9         self.nombre = nombre
10        self.apellido = apellido
11        self.edad = edad
12        self.sexo = sexo
```

```
1 >>> lucia.cambiosexo("M")
2 >>> lucia.sexo
3 "M"
```

Ejemplo

Veamos cómo hacer que nuestra clase tenga más sentido para generar los DNI de las personas:

Ejemplo

Veamos cómo hacer que nuestra clase tenga más sentido para generar los DNI de las personas:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def cambio_sexo(self, nuevo_sexo):
6         self.sexo = nuevo_sexo
7
8     def __init__(self, nombre, apellido, edad, sexo):
9         self.nombre = nombre
10        self.apellido = apellido
11        self.edad = edad
12        self.sexo = sexo
13        self.dni = Persona.dni
14        Persona.dni += 1
```

Ejemplo

Ahora veamos si funciona el creador de DNI consecutivos con personas recién nacidas:

Ejemplo

Ahora veamos si funciona el creador de DNI consecutivos con personas recién nacidas:

```
1 >>> mateo = Persona("Mateo", "Michel", 0, "M")
2 >>> sofia = Persona("Sofia", "Torino", 0, "F")
3 >>> axel = Persona("Axel", "Lopez", 0, "M")
```

Ejemplo

Ahora veamos si funciona el creador de DNI consecutivos con personas recién nacidas:

```
1 >>> mateo = Persona("Mateo", "Michel", 0, "M")
2 >>> sofia = Persona("Sofia", "Torino", 0, "F")
3 >>> axel = Persona("Axel", "Lopez", 0, "M")
```

```
1 >>> mateo.dni
2 57000000
3 >>> sofia.dni
4 57000001
5 >>> axel.dni
6 57000002
7 >>> mateo.pasaporte
8 ANB57000000
9 >>> sofia.pasaporte
10 ANB57000001
```

Herencia

El mecanismo de *herencia* de Python nos permite definir clases derivadas que extienden o modifican el comportamiento de la clase base. La sintaxis para definir una subclase es:

Herencia

El mecanismo de *herencia* de Python nos permite definir clases derivadas que extienden o modifican el comportamiento de la clase base. La sintaxis para definir una subclase es:

```
1 class SubClase( ClaseBase ) :  
2     ...
```

Herencia

El mecanismo de *herencia* de Python nos permite definir clases derivadas que extienden o modifican el comportamiento de la clase base. La sintaxis para definir una subclase es:

```
1 class SubClase( ClaseBase ):
2     ...
```

Por ejemplo,

```
1 class Estudiante(Persona):
2     def __init__(self, nombre, apellido, ..., carrera,\
materias_aprobadas):
3         super().__init__(nombre, apellido, ...)
4         self.carrera = carrera
5         self.materias_aprobadas = materias_aprobadas
6
7     def aprobar(nueva_materia):
8         self.materias_aprobadas.append(nueva_materia)
```

Pregunta

Si definimos la siguiente clase:

```
1 class Persona:
2     def __init__(self, nombre, apellido, edad):
3         self.nombre = nombre
4         self.apellido = apellido
5         self.edad = edad
6
7     def __str__(self):
8         return self.nombre + " " + self.apellido
```

Pregunta

Si definimos la siguiente clase:

```
1 class Persona:
2     def __init__(self, nombre, apellido, edad):
3         self.nombre = nombre
4         self.apellido = apellido
5         self.edad = edad
6
7     def __str__(self):
8         return self.nombre + " " + self.apellido
```

3) ¿Qué imprimiría el siguiente código:

```
1 >>> juan = Persona("Juan", "Tarufetti", 74)
2 >>> print(juan)
```

4) ¿Cómo agregarían un metodo llamado `cambio_nombre` que actualice el nombre de la persona que tramita un cambio de nombre?

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "w")
2 >>> f.write("Estas seran las primeras palabras en el archivo
. \n")
3 >>> f.close()
```

Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "w")
2 >>> f.write("Estas seran las primeras palabras en el archivo
. \n")
3 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "a")
2 >>> f.write("Estas palabras se agregaran al final del
archivo")
3 >>> f.close()
```


Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "w")
2 >>> f.write("Estas seran las primeras palabras en el archivo
. \n")
3 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "a")
2 >>> f.write("Estas palabras se agregaran al final del
archivo")
3 >>> f.close()
```

```
1 >>> with open("archivo.txt") as f:
2 >>>     print(f.read())
3 Estas seran las primeras palabras en el archivo.
4 Estas palabras se agregaran al final del archivo
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Instalación e introducción a pandas

Para poder seguir esta parte de la clase a la par por favor instalen la librería **pandas**

Instalación e introducción a pandas

Para poder seguir esta parte de la clase a la par por favor instalen la librería **pandas**

En la consola:

```
1 >>> pip install pandas
```

O en el Jupyter Notebook:

```
1 import sys
2 !{sys.executable} -m pip install pandas
```

Instalación e introducción a pandas

Para poder seguir esta parte de la clase a la par por favor instalen la librería **pandas**

En la consola:

```
1 >>> pip install pandas
```

O en el Jupyter Notebook:

```
1 import sys
2 !{sys.executable} -m pip install pandas
```

pandas nos permitirá trabajar con datos en formato estructurados. Es decir con tablas de dato como las que estamos acostumbrados a trabajar en Excel o Stata, que contienen la información organizada en filas y columnas.

Importar pandas

pandas es la librería más usada para este tipo de operaciones, tiene funciones y métodos que facilitan mucho el trabajo con tablas de datos (*dataframes*).

En esta clase solo les voy a mostrar como usarla para:

- importar archivos de distintos formatos;
- explorar si se importaron los datos del archivo;

Importar pandas

pandas es la librería más usada para este tipo de operaciones, tiene funciones y métodos que facilitan mucho el trabajo con tablas de datos (*dataframes*).

En esta clase solo les voy a mostrar como usarla para:

- importar archivos de distintos formatos;
- explorar si se importaron los datos del archivo;

Como primer paso, importemos la librería:

```
1 >>> import pandas as pd
```

Abrir un dataframe I

Primer paso, abramos un archivo. Ya vimos una forma de abrir txt, ahora vamos a abrir una tabla con Pandas.

Abrir un dataframe I

Primer paso, abramos un archivo. Ya vimos una forma de abrir txt, ahora vamos a abrir una tabla con Pandas.

Por convención:

- pandas se importa con el alias "pd"
- la base importada se refiere como dataframe, y se abrevia df

Abrir un dataframe I

Primer paso, abramos un archivo. Ya vimos una forma de abrir txt, ahora vamos a abrir una tabla con Pandas.

Por convención:

- pandas se importa con el alias "pd"
- la base importada se refiere como dataframe, y se abrevia df

Pandas tiene funciones para abrir muchos tipos de archivos estructurados. Algunos ejemplos son:

- `pd.read_excel()`
- `pd.read_csv()`
- `pd.read_stata()`
- `pd.read_spss()`

Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel('file_path.xlsx')
```

Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel('file_path.xlsx')
```

Por default Pandas abre la primera hoja del archivo (esté visible o escondida). Para indicar otra hoja:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel('file_path.xlsx')
```

Por default Pandas abre la primera hoja del archivo (esté visible o escondida). Para indicar otra hoja:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Ahora, esa función abre el archivo pero no lo guarda en memoria. Para eso hay que asignarle un nombre de variable:

```
1 >>> import pandas as pd
2 >>> df = pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Abrir un dataframe III

Abrir un .csv es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv('file_path.csv')
```

Abrir un dataframe III

Abrir un .csv es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv('file_path.csv')
```

Al abrir un csv, python intenta inferir el separador. El separador es el símbolo que separa los valores: ",", ";", etc. En general funciona bien, si hay algún inconveniente se puede especificar con:

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv('file_path.csv', sep=";")
```

Abrir un dataframe III

Abrir un .csv es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv('file_path.csv')
```

Al abrir un csv, python intenta inferir el separador. El separador es el símbolo que separa los valores: ",", ";", etc. En general funciona bien, si hay algún inconveniente se puede especificar con:

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv('file_path.csv', sep=";")
```

Abrir un archivo de Stata es similar.

```
1 >>> import pandas as pd
2 >>> df = pd.read_stata('file_path.dta')
```

Nota: Pandas tiene documentación muy completa.

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Para hacer una primera visualización de los datos importados podemos usar:

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Para hacer una primera visualización de los datos importados podemos usar:

- `df.head(N)`: nos muestra las primeras N líneas
- `df.tail(N)`: nos muestra las últimas N líneas
- `df.sample(N)`: muestra aleatoria de N líneas

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Para hacer una primera visualización de los datos importados podemos usar:

- `df.head(N)`: nos muestra las primeras N líneas
- `df.tail(N)`: nos muestra las últimas N líneas
- `df.sample(N)`: muestra aleatoria de N líneas

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv("file_path.csv")
3 >>> df.head(5) #imprime nombres de las columnas y las 5
               primeras lineas
```

```
1 >>> df.tail(2) #igual que head pero empieza desde el fondo
```

```
1 >>> df.sample(10) #muestra aleatoria de n=10
```